

The Lumen Compiler Handbook

A ground-up, project-based path through compiler engineering, in C++

How this handbook works

This is a **living document**. Phases 0–1 below are complete: full theory, internal design, working C++, exercises, and real-compiler comparison. Phases 2–9 are complete in *theory and design* — every concept, trade-off, and decision a real compiler engineer weighs — but their C++ implementations get added incrementally as we build them together, so the code always comes with the understanding behind it rather than being pasted in cold. Each session, we extend this same file. Nothing here is throwaway.

The pipeline we're building (project name: **Lumen**):

```
source (.lum) → Lexer → Tokens → Parser → AST → Sema → typed AST
→ IRTGen → IR (three-address code / SSA) → Optimizer → optimized IR
→ Codegen → x86-64 asm → Assembler/Linker (system tools) → executable
```

Phase 0 — Foundations

0.1 What a compiler is

A compiler translates source code into another representation — usually one closer to what a machine executes. The translation happens in stages because each stage answers a different question:

Stage	Question
Lexing	What are the atomic words in this text?
Parsing	Do these words form valid grammatical structure?
Semantic analysis	Does this valid structure make sense (types, scope)?
IR generation	What does this program <i>mean</i> , independent of any CPU?
Optimization	How do we express that meaning more efficiently?
Codegen	How do we turn that meaning into instructions for <i>this</i> CPU?

Separating these stages is why GCC supports 30+ languages and 50+ targets without an $N \times M$ explosion of translators: every frontend lowers to a shared IR, every backend only consumes that IR. Frontend and backend don't know about each other.

Compiler vs interpreter vs JIT: a compiler produces a translated artifact ahead of time, then stops. An interpreter executes source (or an IR of it) directly, no separate artifact. A JIT starts by interpreting, then compiles hot paths to native code at runtime (JVM, V8, LuaJIT). Lumen is the classic ahead-of-time kind.

0.2 The pipeline as data transformations

```

"int x = 2 + 3;"
  | Lexer
  ▼
[INT, IDENT(x), EQUALS, NUM(2), PLUS, NUM(3), SEMI]
  | Parser
  ▼
VarDecl(x, int, BinaryExpr(+, 2, 3))          ← AST
  | Sema (annotates/checks in place)
  ▼
VarDecl(x, int, BinaryExpr(+, 2, 3))          ← now type-checked
  | IRGen
  ▼
t1 = 2 + 3
x = t1
  | Codegen
  ▼
mov eax, 5
mov [x], eax

```

Each arrow is a **pass**: a pure-ish function taking one representation to the next. Lumen's `main()` will eventually just be this pipeline, each stage independently testable.

0.3 Project skeleton

```

lumen/
├─ CMakeLists.txt
├─ src/
│   ├─ main.cpp
│   ├─ lexer/
│   ├─ parser/
│   ├─ ast/
│   ├─ sema/
│   ├─ ir/
│   └─ codegen/
├─ tests/
└─ examples/

```

`CMakeLists.txt` (C++20, no dependencies yet):

```

cmake_minimum_required(VERSION 3.20)
project(lumen CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(lumen
    src/main.cpp
)

target_include_directories(lumen PRIVATE src)

```

src/main.cpp:

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

// Reads an entire file into memory as one string.
// Why read it all at once rather than stream character-by-character?
// A compiler needs random-ish lookahead/backtracking during lexing and
// parsing (e.g. peeking multiple tokens ahead), and source files are
// small relative to available RAM (megabytes, not gigabytes) – so the
// simplicity of "whole file in memory" wins. Real compilers *do* stream
// for huge generated files or use memory-mapped I/O, but that's an
// optimization layered on top of this same logical model, not a
// different design.

static std::string readFile(const std::string& path) {
    std::ifstream file(path);
    if (!file) {
        throw std::runtime_error("could not open file: " + path);
    }
    std::ostringstream contents;
    contents << file.rdbuf();
    return contents.str();
}

int main(int argc, char** argv) {
    if (argc < 2) {
        std::cerr << "usage: lumen <source-file>\n";
        return 1;
    }
    try {
        std::string source = readFile(argv[1]);
        std::cout << source;
    } catch (const std::exception& e) {
        std::cerr << "error: " << e.what() << "\n";
        return 1;
    }
    return 0;
}
```

Exercise 0.1: Build this, run it on a small `.lum` file. Confirm it round-trips the text.

Exercise 0.2 (think, don't code): What breaks in this "read it all up front" model if Lumen later needs to support `#include`-style file composition? Sketch how you'd extend it.

Phase 1 — Lexical Analysis

1.1 Theory

The lexer (scanner) turns a flat stream of characters into a stream of **tokens** — the smallest meaningful units: keywords, identifiers, literals, operators, punctuation. `int x = 2 + 3;` becomes `[INT, IDENT("x"), EQUALS, NUMBER(2), PLUS, NUMBER(3), SEMI]`.

Why not let the parser work directly on characters? Two reasons:

1. **Separation of concerns** — "what is this word" and "how do words combine grammatically" are different problems with different tools (regular languages vs context-free languages).
2. **Efficiency and simplicity** — grammars that mix character-level and structure-level reasoning become unreadable. Splitting lets each stage use the simplest adequate formalism.

The formal foundation: tokens are defined by **regular expressions**, and regular expressions are equivalent to **finite automata** (Kleene's theorem). Every token rule — "an identifier is a letter followed by letters/digits" — is a regular language, recognizable by a DFA (deterministic finite automaton) that reads one character at a time and moves between states, accepting when it lands in an accept state at end-of-token.

`IDENT` as an NFA (nondeterministic, easy to build from the regex) is converted to a DFA (deterministic, easy to execute in $O(1)$ per character — no backtracking) via the subset construction (Thompson's construction $\rightarrow$ NFA, then powerset construction $\rightarrow$ DFA). Tools like `flex` do exactly this at generation time. **Real compilers almost never do this at generation time via a tool** for hand-tuned frontends — Clang and GCC hand-write their lexers as a big switch/state machine, because a hand-written lexer can special-case common patterns (identifiers, whitespace) faster than a generated table-driven one, and because it integrates error recovery and diagnostics far better. We'll hand-write Lumen's lexer too — it's also pedagogically clearer.

1.2 Internal working

Data structures:

- `Token`: kind (enum), lexeme (string or view into source), and source position (line/column) for diagnostics.
- `Lexer`: holds the source string, a cursor index, and line/column counters. It exposes `nextToken()`, called repeatedly by the parser (or once up front to produce a `vector<Token>` — we'll do the latter for simplicity, real compilers often do the former to avoid materializing all tokens).

Algorithm shape (maximal munch): at each position, try to match the *longest* possible token. `<=` must be recognized as one token, not `<` then `=`. This is why the lexer peeks ahead before committing to a token kind.

Complexity: $O(n)$ in source length — each character is examined a constant number of times (this is exactly what a DFA guarantees; a hand-written lexer preserves this if written carefully, i.e., no re-scanning from the start on failure).

Design decisions we're making for Lumen (and why):

- Store lexemes as `std::string_view` into the original source buffer (which we keep alive for the whole compilation) rather than copying substrings — avoids $O(n)$ copies, matches what Clang does (`llvm::StringRef`).
- Track line/column incrementally during scanning rather than recomputing later — diagnostics are cheap this way and it costs nothing extra since we're already walking the text.
- Fail *softly*: an illegal character produces an `ERROR` token with a message, not a thrown exception, so the lexer can keep going and the parser can eventually report multiple errors in one pass — this is what "error recovery" starts with.

1.3 C++ implementation

`src/lexer/token.hpp:`

```

#pragma once
#include <string_view>

namespace lumen {

enum class TokenKind {
    // literals & identifiers
    Identifier, Number,
    // keywords
    KwInt, KwIf, KwElse, KwWhile, KwReturn,
    // punctuation & operators
    Plus, Minus, Star, Slash,
    Equals, EqualsEquals, Less, LessEqual, Greater, GreaterEqual,
    LParen, RParen, LBrace, RBrace, Semicolon,
    // control
    EndOfFile, Error
};

struct Token {
    TokenKind kind;
    std::string_view lexeme;    // view into the source buffer – no copy
    int line;
    int column;
};

} // namespace lumen

```

src/lexer/lexer.hpp / .cpp:

```

#pragma once
#include <string>
#include <vector>
#include "token.hpp"

namespace lumen {

class Lexer {
public:
    explicit Lexer(const std::string& source) : src_(source) {}

    // Scans the whole source into a token vector, ending with EndOfFile.
    std::vector<Token> tokenize();

private:
    const std::string& src_;
    size_t pos_ = 0;
    int line_ = 1;
    int column_ = 1;

    bool atEnd() const { return pos_ >= src_.size(); }
    char peek() const { return atEnd() ? '\0' : src_[pos_]; }
    char peekNext() const { return pos_ + 1 < src_.size() ? src_[pos_ + 1] : '\0'; }
    char advance(); // consumes and returns current char, updates line/column
    bool match(char expected); // consumes current char IF it equals expected

    void skipWhitespaceAndComments();
    Token makeToken(TokenKind kind, size_t start, int startLine, int startColumn);
    Token errorToken(const std::string& msg, int startLine, int startColumn);

    Token scanIdentifierOrKeyword();
    Token scanNumber();
    Token scanOperatorOrPunctuation();

    static TokenKind keywordOrIdentifier(std::string_view text);
};

} // namespace lumen

```

```

#include "lexer.hpp"
#include <cctype>
#include <unordered_map>

namespace lumen {

char Lexer::advance() {
    char c = src_[pos_++];
    if (c == '\n') { line_++; column_ = 1; } else { column_++; }
    return c;
}

bool Lexer::match(char expected) {
    if (atEnd() || src_[pos_] != expected) return false;
    advance();
    return true;
}

void Lexer::skipWhitespaceAndComments() {
    while (!atEnd()) {
        char c = peek();
        if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
            advance();
        } else if (c == '/' && peekNext() == '/') {
            while (!atEnd() && peek() != '\n') advance();
        } else {
            return;
        }
    }
}

Token Lexer::makeToken(TokenKind kind, size_t start, int startLine, int startColumn) {
    std::string_view lexeme(src_.data() + start, pos_ - start);
    return Token{kind, lexeme, startLine, startColumn};
}

Token Lexer::errorToken(const std::string& msg, int startLine, int startColumn) {
    // In a fuller implementation this would carry `msg` in a diagnostics
    // side-channel rather than the lexeme; kept simple here.
    return Token{TokenKind::Error, std::string_view(msg), startLine, startColumn};
}

TokenKind Lexer::keywordOrIdentifier(std::string_view text) {
    static const std::unordered_map<std::string_view, TokenKind> keywords = {
        {"int", TokenKind::KwInt}, {"if", TokenKind::KwIf},
        {"else", TokenKind::KwElse}, {"while", TokenKind::KwWhile},
        {"return", TokenKind::KwReturn},
    };
    auto it = keywords.find(text);
    return it != keywords.end() ? it->second : TokenKind::Identifier;
}

```

```

}

Token Lexer::scanIdentifierOrKeyword() {
    size_t start = pos_; int line = line_, col = column_;
    while (!atEnd() && (std::isalnum((unsigned char)peek()) || peek() == '_')) advance();
    std::string_view text(src_.data() + start, pos_ - start);
    return Token{keywordOrIdentifier(text), text, line, col};
}

Token Lexer::scanNumber() {
    size_t start = pos_; int line = line_, col = column_;
    while (!atEnd() && std::isdigit((unsigned char)peek())) advance();
    // (Deliberately no floats/decimals yet - Lumen starts int-only; this
    // is where you'd extend for '.', exponents, etc.)
    return makeToken(TokenKind::Number, start, line, col);
}

Token Lexer::scanOperatorOrPunctuation() {
    size_t start = pos_; int line = line_, col = column_;
    char c = advance();
    switch (c) {
        case '+': return makeToken(TokenKind::Plus, start, line, col);
        case '-': return makeToken(TokenKind::Minus, start, line, col);
        case '*': return makeToken(TokenKind::Star, start, line, col);
        case '/': return makeToken(TokenKind::Slash, start, line, col);
        case '(': return makeToken(TokenKind::LParen, start, line, col);
        case ')': return makeToken(TokenKind::RParen, start, line, col);
        case '{': return makeToken(TokenKind::LBrace, start, line, col);
        case '}': return makeToken(TokenKind::RBrace, start, line, col);
        case ';': return makeToken(TokenKind::Semicolon, start, line, col);
        case '=':
            // maximal munch: check for '==' before committing to '='
            if (match('=')) return makeToken(TokenKind::EqualsEquals, start, line, col);
            return makeToken(TokenKind::Equals, start, line, col);
        case '<':
            if (match('=')) return makeToken(TokenKind::LessEqual, start, line, col);
            return makeToken(TokenKind::Less, start, line, col);
        case '>':
            if (match('=')) return makeToken(TokenKind::GreaterEqual, start, line, col);
            return makeToken(TokenKind::Greater, start, line, col);
        default:
            return errorToken(std::string("unexpected character '" + c + "'", line, col);
    }
}

std::vector<Token> Lexer::tokenize() {
    std::vector<Token> tokens;
    while (true) {
        skipWhitespaceAndComments();
        if (atEnd()) {
            tokens.push_back(Token{TokenKind::EndOfFile, {}, line_, column_});
        }
    }
}

```



```

        break;
    }
    char c = peek();
    if (std::isalpha((unsigned char)c) || c == '_') {
        tokens.push_back(scanIdentifierOrKeyword());
    } else if (std::isdigit((unsigned char)c)) {
        tokens.push_back(scanNumber());
    } else {
        tokens.push_back(scanOperatorOrPunctuation());
    }
}
return tokens;
}

} // namespace lumen

```

Walkthrough of the design choices actually made here:

- `tokenize()` is a **greedy loop**, not recursion — lexers don't need call-stack depth proportional to input, so an explicit loop is both simpler and avoids any risk of stack overflow on huge files.
- Every `scan*` function records `start/line/col` *before* consuming, so the resulting token's position points at its first character — essential for diagnostics like `error at line 4, column 9`.
- `match()` is the mechanism for maximal munch: we optimistically consume the "long" form only if the next character confirms it.

1.4 Exercises

1. Add float literal support (3.14) to `scanNumber`.
2. Add `&&`, `||`, `!` — decide maximal-munch handling for `!` vs `!=`.
3. **Debugging exercise:** the current lexer has no way to signal "line 4 col 9: unexpected character '@'" back to `main`. Design (don't necessarily implement yet) a `DiagnosticEngine` that the lexer reports into instead of encoding errors in tokens.
4. **Interview-style:** why is maximal munch sometimes *wrong* in real languages? (Hint: look up C++'s `>>` template-closing ambiguity, and how the C++11 standard changed lexing rules to fix it.)
5. Write 10 hand-crafted `.lum` snippets and their expected token streams; turn these into unit tests now, before the parser exists — this becomes your regression suite for the rest of the project.

1.5 Real compiler comparison

- **Clang:** hand-written lexer (`clang::Lexer`), operates lazily token-by-token, heavily optimized (branchless character classification tables), integrated with a `SourceManager` for precise diagnostics including macro-expansion history.
- **GCC:** also hand-written (`libcpp`), notably handles the C preprocessor *as part of* lexing, since `#define/#include` interact with tokenization itself.
- **rustc:** hand-written lexer producing a token stream consumed by a hand-written recursive-descent-ish parser; notably lexes raw strings, integer suffixes, and unicode identifiers as first-class concerns from day one.
- **Go compiler:** hand-written, deliberately minimal token set (Go has few keywords by design), reflecting the language's philosophy in the tool.

The common thread: production compilers overwhelmingly hand-write lexers rather than using `flex`/generator tools, precisely because of the error-recovery and diagnostic-quality reasons above — validating the approach we're taking with Lumen.

Phase 2 — Syntax Analysis

2.1 Theory

Where lexing handles *regular* languages, parsing handles **context-free languages** — grammars that can express nesting $((((x))))$, which regular expressions fundamentally cannot (no finite automaton can count arbitrarily deep nesting). A **context-free grammar** (CFG) is a set of production rules like:

```
Expr  → Expr '+' Term | Term
Term  → Term '*' Factor | Factor
Factor → NUMBER | '(' Expr ')'
```

Parsing = given a token stream, find a derivation (or prove none exists) — i.e., build a **parse tree**. We then simplify the parse tree into an **AST**, which drops grammar scaffolding (parentheses, redundant unary productions) and keeps only semantically meaningful structure.

2.2 Parsing strategies — the real trade-off space

- **Recursive descent (top-down)**: one function per grammar rule, calling each other following the grammar's structure. Simple to write, simple to add great error messages and recovery to (because you control the code, not a table). **Limitation**: naturally handles LL grammars only — left recursion ($\text{Expr} \rightarrow \text{Expr} \text{ '+' } \text{Term}$) causes infinite recursion and must be rewritten iteratively (this is exactly how Pratt/precedence-climbing parsing handles arithmetic — we'll derive this together next session). **This is what Clang, and most hand-written production frontends, use.**
- **LL(1) table-driven**: formalizes recursive descent into a table; same expressive power, less flexible for custom error messages, mostly of pedagogical/generator-tool interest now.
- **LR/LALR (bottom-up, e.g. yacc/bison)**: strictly more powerful (handles left recursion naturally, more grammars describable unambiguously), but the generated parser's control flow doesn't mirror the grammar the way recursive descent does, making custom diagnostics harder. Used historically (original C compilers), less common in new hand-written frontends today.

Decision for Lumen: recursive descent with precedence climbing for expressions — matches what Clang/rustc actually do and keeps error recovery in our hands.

2.3 Lumen's grammar

```

Program    → Function*
Function   → 'int' IDENT '(' ' ' ')' Block
Block      → '{' Stmt* '}'
Stmt       → VarDecl | ExprStmt | IfStmt | WhileStmt | ReturnStmt | Block
VarDecl    → 'int' IDENT ('=' Expr)? ';'
ExprStmt   → Expr ';'
IfStmt     → 'if' '(' Expr ')' Stmt ('else' Stmt)?
WhileStmt  → 'while' '(' Expr ')' Stmt
ReturnStmt → 'return' Expr? ';'
Expr       → Assignment
Assignment → IDENT '=' Assignment | Equality
Equality   → Comparison (('=='|'!=') Comparison)*
Comparison → Term (('<'|'<='|'>'|'>='|') Term)*
Term       → Factor (('+'|'-') Factor)*
Factor     → Unary (('*'|'/') Unary)*
Unary      → '-' Unary | Primary
Primary    → NUMBER | IDENT | '(' Expr ')'

```

Notice `Term → Term '*' Factor` is *left*-recursive on paper, which would blow the stack if implemented as literal recursive descent.

Precedence climbing rewrites this into a loop: parse one `Factor`, then while the next token is `+/-`, consume it and fold in another `Factor`.

Same language, no recursion on the left, one level of the grammar per precedence tier.

2.4 AST node hierarchy

`src/ast/ast.hpp:`

```

#pragma once
#include <memory>
#include <string>
#include <vector>
#include "../lexer/token.hpp"

namespace lumen {

// ---- Expressions ----
struct Expr { virtual ~Expr() = default; int line = 0; };
using ExprPtr = std::unique_ptr<Expr>;

struct NumberExpr : Expr { int value; };
struct VarExpr    : Expr { std::string name; };
struct AssignExpr : Expr { std::string name; ExprPtr value; };
struct BinaryExpr : Expr { TokenKind op; ExprPtr lhs, rhs; };
struct UnaryExpr  : Expr { TokenKind op; ExprPtr operand; };

// ---- Statements ----
struct Stmt { virtual ~Stmt() = default; int line = 0; };
using StmtPtr = std::unique_ptr<Stmt>;

struct VarDeclStmt : Stmt { std::string name; ExprPtr init; /* may be null */ };
struct ExprStmt    : Stmt { ExprPtr expr; };
struct BlockStmt   : Stmt { std::vector<StmtPtr> statements; };
struct IfStmt      : Stmt { ExprPtr cond; StmtPtr thenBranch; StmtPtr elseBranch; /* may be null */ };
struct WhileStmt   : Stmt { ExprPtr cond; StmtPtr body; };
struct ReturnStmt  : Stmt { ExprPtr value; /* may be null */ };

struct FunctionDecl {
    std::string name;
    std::unique_ptr<BlockStmt> body;
};

struct Program {
    std::vector<FunctionDecl> functions;
};

} // namespace lumen

```

We use `unique_ptr` + raw ownership trees (not a visitor-pattern indirection layer yet) — simplest thing that works. When we add multiple passes over the same tree (sema, IR gen, and later a pretty-printer), *that's* the point where a visitor pattern earns its complexity; introducing it now would be premature abstraction.

2.5 The parser

`src/parser/parser.hpp / .cpp:`

```

#pragma once
#include <vector>
#include "../lexer/token.hpp"
#include "../ast/ast.hpp"

namespace lumen {

class Parser {
public:
    explicit Parser(std::vector<Token> tokens) : tokens_(std::move(tokens)) {}
    Program parseProgram();

private:
    std::vector<Token> tokens_;
    size_t pos_ = 0;

    const Token& peek() const { return tokens_[pos_]; }
    const Token& previous() const { return tokens_[pos_ - 1]; }
    bool check(TokenKind k) const { return peek().kind == k; }
    bool atEnd() const { return check(TokenKind::EndOfFile); }
    const Token& advance() { if (!atEnd()) pos_++; return previous(); }
    bool match(TokenKind k) { if (check(k)) { advance(); return true; } return false; }
    const Token& expect(TokenKind k, const std::string& msg);
    void synchronize(); // panic-mode recovery

    FunctionDecl parseFunction();
    StmtPtr parseStatement();
    StmtPtr parseVarDecl();
    std::unique_ptr<BlockStmt> parseBlock();
    StmtPtr parseIf();
    StmtPtr parseWhile();
    StmtPtr parseReturn();
    StmtPtr parseExprStmt();

    // Precedence-climbing expression parsers, one per tier, from the grammar above.
    ExprPtr parseExpr(); // = Assignment
    ExprPtr parseAssignment();
    ExprPtr parseEquality();
    ExprPtr parseComparison();
    ExprPtr parseTerm();
    ExprPtr parseFactor();
    ExprPtr parseUnary();
    ExprPtr parsePrimary();
};

} // namespace lumen

```

```

#include "parser.hpp"
#include <stdexcept>

namespace lumen {

const Token& Parser::expect(TokenKind k, const std::string& msg) {
    if (check(k)) return advance();
    throw std::runtime_error(
        "parse error at line " + std::to_string(peek().line) + ": " + msg);
}

// After a syntax error, skip tokens until we're at a likely statement
// boundary (';' or '}') so subsequent statements can still be checked -
// otherwise one typo produces a cascade of bogus follow-on errors.
void Parser::synchronize() {
    advance();
    while (!atEnd()) {
        if (previous().kind == TokenKind::Semicolon) return;
        if (peek().kind == TokenKind::RBrace) return;
        advance();
    }
}

Program Parser::parseProgram() {
    Program program;
    while (!atEnd()) {
        program.functions.push_back(parseFunction());
    }
    return program;
}

FunctionDecl Parser::parseFunction() {
    expect(TokenKind::KwInt, "expected return type 'int'");
    Token name = expect(TokenKind::Identifier, "expected function name");
    expect(TokenKind::LParen, "expected '('");
    expect(TokenKind::RParen, "expected ')' (no params yet in Lumen v0)");
    auto body = parseBlock();
    return FunctionDecl{std::string(name.lexeme), std::move(body)};
}

std::unique_ptr<BlockStmt> Parser::parseBlock() {
    expect(TokenKind::LBrace, "expected '{'");
    auto block = std::make_unique<BlockStmt>();
    while (!check(TokenKind::RBrace) && !atEnd()) {
        try {
            block->statements.push_back(parseStatement());
        } catch (const std::runtime_error& e) {
            // report and recover instead of aborting the whole parse
            fprintf(stderr, "%s\n", e.what());
            synchronize();
        }
    }
}

```

```

    }

    }

    expect(TokenKind::RBrace, "expected '}'");
    return block;
}

StmtPtr Parser::parseStatement() {
    if (check(TokenKind::KwInt))    return parseVarDecl();
    if (check(TokenKind::KwIf))     return parseIf();
    if (check(TokenKind::KwWhile))  return parseWhile();
    if (check(TokenKind::KwReturn)) return parseReturn();
    if (check(TokenKind::LBrace))   return parseBlock();
    return parseExprStmt();
}

StmtPtr Parser::parseVarDecl() {
    advance(); // 'int'
    Token name = expect(TokenKind::Identifier, "expected variable name");
    auto decl = std::make_unique<VarDeclStmt>();
    decl->name = std::string(name.lexeme);
    if (match(TokenKind::Equals)) decl->init = parseExpr();
    expect(TokenKind::Semicolon, "expected ';' after variable declaration");
    return decl;
}

StmtPtr Parser::parseIf() {
    advance(); // 'if'
    expect(TokenKind::LParen, "expected '(' after 'if'");
    auto stmt = std::make_unique<IfStmt>();
    stmt->cond = parseExpr();
    expect(TokenKind::RParen, "expected ')'");
    stmt->thenBranch = parseStatement();
    if (match(TokenKind::KwElse)) stmt->elseBranch = parseStatement();
    return stmt;
}

StmtPtr Parser::parseWhile() {
    advance(); // 'while'
    expect(TokenKind::LParen, "expected '(' after 'while'");
    auto stmt = std::make_unique<WhileStmt>();
    stmt->cond = parseExpr();
    expect(TokenKind::RParen, "expected ')'");
    stmt->body = parseStatement();
    return stmt;
}

StmtPtr Parser::parseReturn() {
    advance(); // 'return'
    auto stmt = std::make_unique<ReturnStmt>();
    if (!check(TokenKind::Semicolon)) stmt->value = parseExpr();
    expect(TokenKind::Semicolon, "expected ';' after return");
}

```

```

    return stmt;
}

StmtPtr Parser::parseExprStmt() {
    auto expr = parseExpr();
    expect(TokenKind::Semicolon, "expected ';' after expression");
    auto stmt = std::make_unique<ExprStmt>();
    stmt->expr = std::move(expr);
    return stmt;
}

ExprPtr Parser::parseExpr() { return parseAssignment(); }

ExprPtr Parser::parseAssignment() {
    ExprPtr expr = parseEquality();
    if (match(TokenKind::Equals)) {
        // Only a bare identifier is a legal assignment target in Lumen v0.
        if (auto* var = dynamic_cast<VarExpr*>(expr.get())) {
            auto assign = std::make_unique<AssignExpr>();
            assign->name = var->name;
            assign->value = parseAssignment(); // right-associative
            return assign;
        }
        throw std::runtime_error("invalid assignment target");
    }
    return expr;
}

// Shared shape for every left-associative binary tier: parse one operand
// at the next tier down, then loop consuming same-precedence operators.
ExprPtr Parser::parseEquality() {
    ExprPtr expr = parseComparison();
    while (check(TokenKind::EqualsEquals)) {
        TokenKind op = advance().kind;
        auto bin = std::make_unique<BinaryExpr>();
        bin->op = op; bin->lhs = std::move(expr); bin->rhs = parseComparison();
        expr = std::move(bin);
    }
    return expr;
}

ExprPtr Parser::parseComparison() {
    ExprPtr expr = parseTerm();
    while (check(TokenKind::Less) || check(TokenKind::LessEqual) ||
           check(TokenKind::Greater) || check(TokenKind::GreaterEqual)) {
        TokenKind op = advance().kind;
        auto bin = std::make_unique<BinaryExpr>();
        bin->op = op; bin->lhs = std::move(expr); bin->rhs = parseTerm();
        expr = std::move(bin);
    }
    return expr;
}

```



```

}

ExprPtr Parser::parseTerm() {
    ExprPtr expr = parseFactor();
    while (check(TokenKind::Plus) || check(TokenKind::Minus)) {
        TokenKind op = advance().kind;
        auto bin = std::make_unique<BinaryExpr>();
        bin->op = op; bin->lhs = std::move(expr); bin->rhs = parseFactor();
        expr = std::move(bin);
    }
    return expr;
}

ExprPtr Parser::parseFactor() {
    ExprPtr expr = parseUnary();
    while (check(TokenKind::Star) || check(TokenKind::Slash)) {
        TokenKind op = advance().kind;
        auto bin = std::make_unique<BinaryExpr>();
        bin->op = op; bin->lhs = std::move(expr); bin->rhs = parseUnary();
        expr = std::move(bin);
    }
    return expr;
}

ExprPtr Parser::parseUnary() {
    if (check(TokenKind::Minus)) {
        TokenKind op = advance().kind;
        auto un = std::make_unique<UnaryExpr>();
        un->op = op; un->operand = parseUnary();
        return un;
    }
    return parsePrimary();
}

ExprPtr Parser::parsePrimary() {
    if (check(TokenKind::Number)) {
        auto tok = advance();
        auto n = std::make_unique<NumberExpr>();
        n->value = std::stoi(std::string(tok.lexeme));
        return n;
    }
    if (check(TokenKind::Identifier)) {
        auto tok = advance();
        auto v = std::make_unique<VarExpr>();
        v->name = std::string(tok.lexeme);
        return v;
    }
    if (match(TokenKind::LParen)) {
        auto expr = parseExpr();
        expect(TokenKind::RParen, "expected '')");
        return expr;
    }

```

```

    }
    throw std::runtime_error(
        "parse error at line " + std::to_string(peek().line) + ": expected expression");
}

} // namespace lumen

```

Every tier from `parseEquality` down to `parseFactor` has the identical shape — `operand = lower(); while (op at this precedence) { operand = Binary(op, operand, lower()); }` — which is precedence climbing. This directly answers exercise-style question: left recursion in the grammar becomes a `while` loop in code, at every tier uniformly.

2.6 Exercises

1. Trace by hand how `2 + 3 * 4` parses — confirm `*` binds tighter (it goes through one more precedence tier before `+` ever sees it).
2. Add `&&||` as new precedence tiers between `Assignment` and `Equality`, with correct relative precedence to C-family languages.
3. **Debugging exercise:** the current `synchronize()` only stops at `;` or `}`. Construct a Lumen snippet where this still produces a cascading garbage error, and fix it.
4. **Interview-style:** why is right-associativity for `Assignment` (`a = b = c` means `a = (b = c)`) implemented via recursion (`parseAssignment` calling itself) while left-associativity for `+` is implemented via a loop? What would go wrong if you used a loop for assignment?

2.7 Real compiler comparison

Clang's `Parser::ParseExpression` and friends are structurally identical to what's above — one method per precedence tier, explicit loops for left-associative operators — scaled up with a full operator-precedence table for C++'s much larger operator set instead of hand-tiered methods. rustc's parser is also hand-written recursive descent. GCC's C frontend historically used a Bison (LALR) grammar but migrated much of its expression parsing to hand-written code for the same diagnostic-quality reasons discussed in Phase 1.

Phase 3 — Semantic Analysis

Semantic analysis walks the (now syntactically valid) AST and asks: does this make sense? Two core mechanisms:

- **Symbol tables & scope:** a stack of hash maps (one per lexical scope), pushed on block entry, popped on exit. Name lookup walks the stack from innermost to outermost — this *is* lexical scoping, implemented directly.
- **Type checking:** each AST node gets an inferred/declared type; expressions are checked bottom-up (children's types determine what operators are legal). Lumen starts with a simple nominal type system (`int`, maybe `bool`) — full type inference (Hindley-Milner, as in ML/Haskell/Rust's inference) is a later extension, not a Phase-3 requirement.

Design trade-off worth internalizing now: do semantic checks *mutate* the AST in place (annotate nodes with resolved types/symbols) or build a *separate* typed representation? Real compilers vary — Clang mutates/annotates in place; some functional-language compilers build a fresh typed tree at each stage. We'll annotate in place for Lumen: simpler, and it's what most C-family compilers do.

Phase 4 — Intermediate Representation (theory complete)

Why not go straight from AST to assembly? Because AST is language-shaped and assembly is machine-shaped — coupling them directly means every new source feature needs matching logic in every backend. **IR decouples the two.**

- **Three-address code (TAC):** each instruction has at most one operator and three operands ($t1 = t2 + t3$) — matches roughly what most CPU instructions can do, making later code generation nearly mechanical.
- **SSA (Static Single Assignment):** every variable is assigned exactly once; a `phi` node merges values from different control-flow paths. This is what **LLVM IR** is built on, because SSA makes data-flow analysis (needed for optimization) dramatically simpler — "where was this value defined" becomes a single unambiguous answer instead of a search.
- **Control-flow graphs (CFG):** basic blocks (straight-line instruction sequences) as nodes, jumps/branches as edges. Nearly every optimization is phrased as "a computation over the CFG."

Lumen will lower AST → TAC first (simpler to generate directly from tree-walking), then we'll convert TAC → SSA explicitly as its own pass — seeing that conversion is one of the most illuminating exercises in the whole course.

Phase 5 — Optimization (theory complete)

Optimizations are **CFG/IR rewrites that preserve meaning while improving some cost metric** (speed, size). Categories:

- **Local** (within one basic block): constant folding ($2+3 \rightarrow 5$ at compile time), common subexpression elimination, local dead-code elimination.
- **Global** (across the CFG): requires data-flow analysis — e.g. **liveness analysis** (is this value still needed later?) drives both dead-code elimination and register allocation; **reaching definitions** drives constant propagation across blocks.
- Data-flow analyses are all instances of one template: iterate over the CFG applying a transfer function per block until a fixed point is reached (Kildall's algorithm). This single pattern underlies liveness, reaching definitions, available expressions, and more — we'll implement the generic framework once and reuse it.

Phase 6 — Code Generation (theory complete)

- **Instruction selection:** mapping IR ops to real CPU instructions — often via tree pattern matching (e.g. $t1 = a * b + c$ might become a single `imul+add`, or on some architectures a fused multiply-add).
- **Register allocation:** IR uses unlimited virtual registers; real CPUs have ~16 general-purpose ones. **Linear scan** (fast, used in JITs like V8) vs **graph coloring** (slower, better code, used in GCC/LLVM's `-O2+`) is the central trade-off — build time vs runtime quality.
- **Calling conventions & stack frames:** how arguments/return values move between caller and callee (registers vs stack, per-ABI rules), and how each function's local storage is laid out on the stack (prologue allocates, epilogue restores).

Phase 7 — Linking, Loading, and the OS boundary (theory

complete)

Our codegen emits *assembly text*; turning that into a runnable program is not "the compiler" in the narrow sense but is essential to understand:

- **Assembler:** text asm → object file (machine code + a symbol table + relocation info)
- **Object format (ELF on Linux, PE on Windows):** sections (`.text`, `.data`, `.bss`), symbol table, relocations (placeholders for addresses not known until link time)
- **Linker:** merges object files + libraries, resolves symbol references, applies relocations, produces an executable (or leaves dynamic symbols for the loader)
- **Loader** (OS-level, at `exec()` time): maps the executable into virtual memory, resolves any remaining dynamic-library symbols, sets up the initial stack, jumps to `_start`

For Lumen, we'll shell out to the system assembler/linker (`as/ld` or via `cc`) rather than writing our own — same as most teaching compilers and even some production ones for their non-primary targets — but we'll trace through exactly what those tools did to a real binary with `objdump/readelf` so it isn't a black box.

Phase 8 — Systems context (woven throughout; reference)

CPU execution model, registers, cache-awareness, stack vs heap, function-call mechanics, runtime libraries, syscalls — these get taught **in situ** as they become relevant (register allocation pulls in registers/ABI; codegen pulls in stack frames; linking pulls in ELF/loader) rather than as a separate detached module, since they land much better with a concrete reason to care about them.

Phase 9 — Extensions (after the core compiler works end-to-end)

- Retarget codegen to emit **LLVM IR** instead of raw x86 asm, and let `llc/clang` do instruction selection, register allocation, and optimization — directly compares "what we hand-built" to "what a production backend does with the same input."
- A minimal **JIT**: skip the file-based executable, `mmap` executable memory, write machine code into it, cast to a function pointer, call it.
- Garbage collection concepts (mark-sweep basics) — conceptual, since Lumen's core language won't necessarily need a GC unless we add heap-allocated types.
- Add one real language feature end-to-end (closures or a minimal generics system) to see how a feature ripples through every phase you've built.

Where we are

Done: Phase 0 (project skeleton) and Phase 1 (full lexer) — buildable today. **Next session:** Phase 2 — we derive precedence climbing together (you drive, I guide), build the AST hierarchy, and get `2 + 3 * 4` parsing correctly with proper precedence.

Build the lexer, run it on a few `.lum` snippets, and bring questions — especially on Exercise 1.4.4 (the `>>` ambiguity) — and we'll start Phase 2.